WAZE FURTHER EDA ANALYSIS

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# Load the dataset into a dataframe
df = pd.read_csv('waze_dataset.csv')
# The dataframe 'df' is already loaded in the environment, so we can display its head
df.head()
```

...	...	...	...	...	total_se...	n_days_after_onboardi...	total_navigations_fav1	total_navigations_fav2	driven_km_...	duration_minutes_drives	activit...
0	0	retained	283	226	296.7482729	2276	208	0	2628.845068	1985.775061	
1	1	retained	133	107	326.8965962	1225	19	64	13715.92055	3160.472914	
2	2	retained	114	95	135.5229263	2651	0	0	3059.148818	1610.735904	
3	3	retained	49	40	67.58922127	15	322	7	913.5911231	587.1965423	
4	4	retained	84	68	168.2470201	1562	166	5	3950.202008	1219.555924	

Rows: 5 Expand

```
df.size
```

194987

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 13 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   ID                       14999 non-null  int64
 1   label                    14299 non-null  object
 2   sessions                 14999 non-null  int64
 3   drives                   14999 non-null  int64
 4   total_sessions           14999 non-null  float64
 5   n_days_after_onboarding  14999 non-null  int64
 6   total_navigations_fav1   14999 non-null  int64
 7   total_navigations_fav2   14999 non-null  int64
 8   driven_km_drives         14999 non-null  float64
 9   duration_minutes_drives  14999 non-null  float64
 10  activity_days            14999 non-null  int64
 11  driving_days             14999 non-null  int64
 12  device                   14999 non-null  object
dtypes: float64(3), int64(8), object(2)
memory usage: 1.5+ MB
```

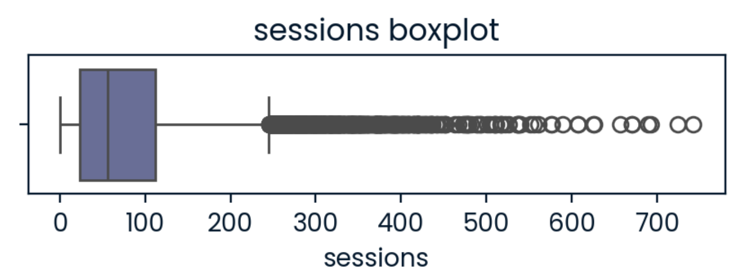
```python
df.describe()
```

	ID	sessions	drives	total_se...	n_days_after_onboardi...	total_navigations_fav1	total_navigations_fav2	driven_km_...	duration_minutes_drives
count	14999	14999	14999	14999	14999	14999	14999	14999	14999
mean	7499	80.633775585	67.2811520768	189.964446824	1749.8377891859	121.6059737316	29.6725115008	4039.3409208165	1860.9760121294
std	4329.9826789492	80.6990648469	65.9138724189	136.4051284776	1008.5138761173	148.1215436251	45.3946507312	2502.1493337965	1446.7022875633
min	0	0	0	0.2202109438	4	0	0	60.44125046	18.28208247
25%	3749.5	23	20	90.66115631	878	9	0	2212.6006065	835.9962599
50%	7499	56	48	159.5681147	1741	71	9	3493.858085	1478.2498559
75%	11248.5	112	93	254.19234075	2623.5	178	43	5289.861262	2464.3626632
max	14998	743	596	1216.154633	3500	1236	415	21183.40189	15851.72716

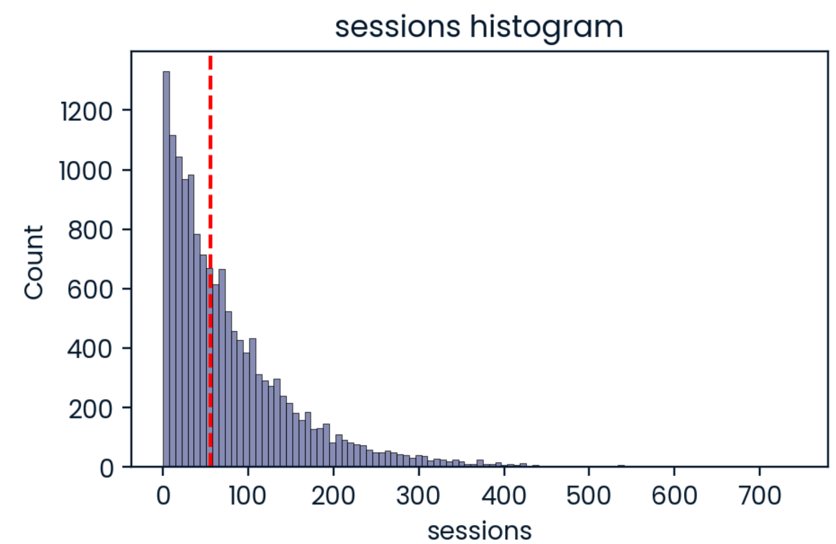
Rows: 8

Expand

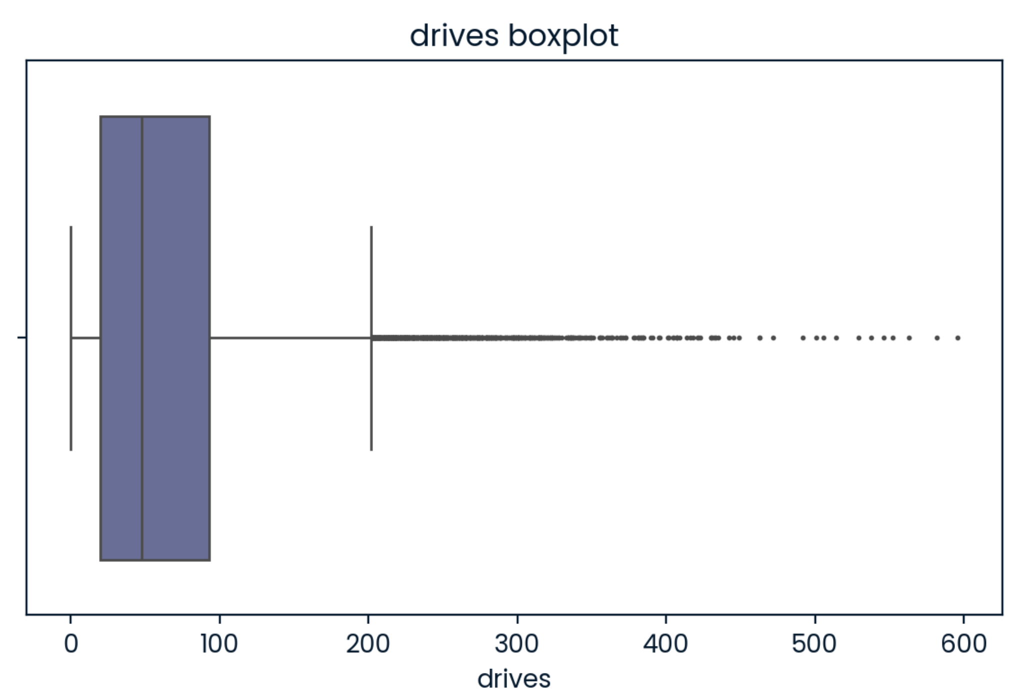
```python
# Box plot
plt.figure(figsize=(5,1))
plt.title('sessions boxplot')
sns.boxplot(x=df['sessions']);
```



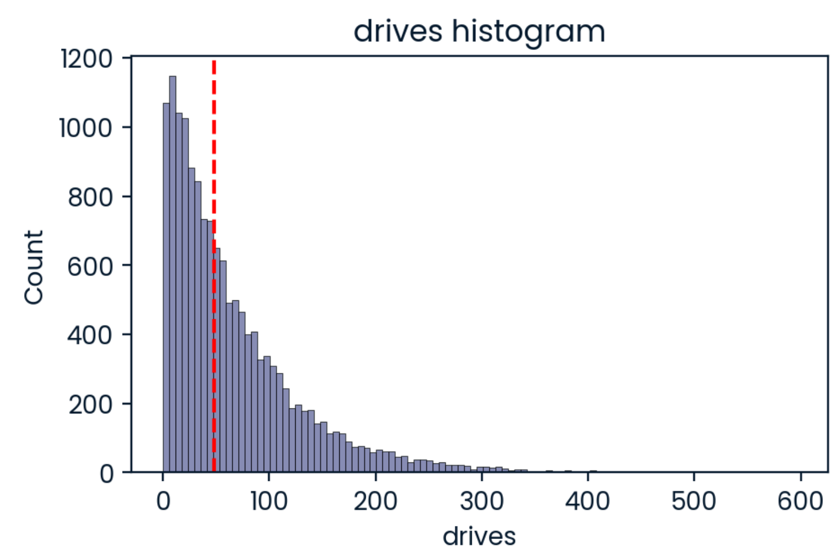
```python
# Histogram
plt.figure(figsize=(5,3))
plt.title('sessions histogram')
sns.histplot(x=df['sessions'])
median = df['sessions'].median()
plt.axvline(median, color='red', linestyle='--');
```



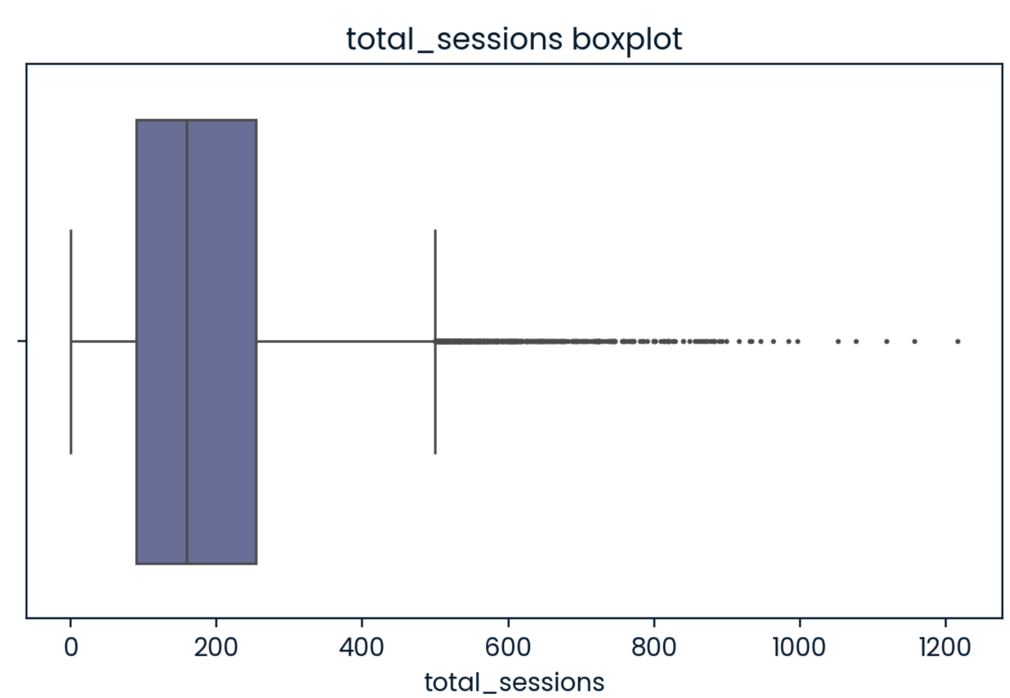

```python
# Box plot
plt.figure(figsize=(7,4))
plt.title('drives boxplot')
sns.boxplot(x=df['drives'],fliersize=1);
```



```
# Histogram
plt.figure(figsize=(5,3))
plt.title('drives histogram')
sns.histplot(x=df['drives'])
median = df['drives'].median()
plt.axvline(median, color='red', linestyle='--');
```

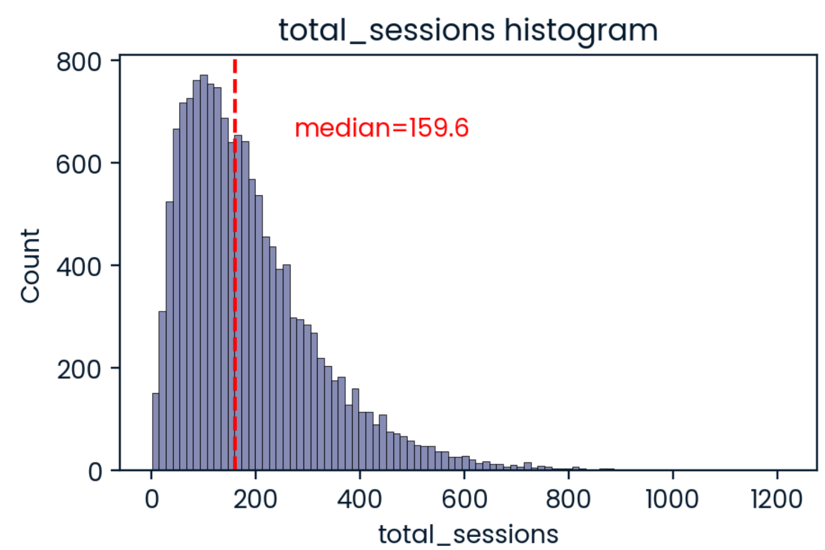



```
# Box plot
plt.figure(figsize=(7,4))
plt.title('total_sessions boxplot')
sns.boxplot(x=df['total_sessions'],fliersize=1);
```

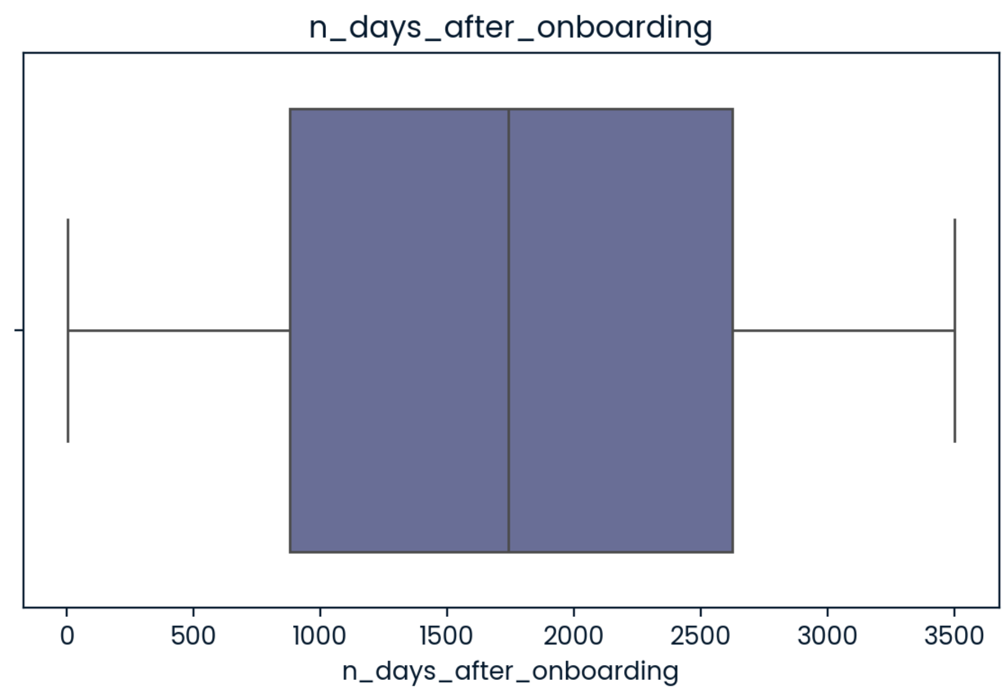


```python
def histogrammer(column_str, median_text=True, **kwargs):    # **kwargs = any keyword arguments
                                                             # from the sns.histplot() function
    median=round(df[column_str].median(), 1)
    plt.figure(figsize=(5,3))
    ax = sns.histplot(x=df[column_str], **kwargs)            # Plot the histogram
    plt.axvline(median, color='red', linestyle='--')         # Plot the median line
    if median_text==True:                                    # Add median text unless set to False
        ax.text(0.25, 0.85, f'median={median}', color='red',
            ha='left', va='top', transform=ax.transAxes)
    else:
        print('Median:', median)
    plt.title(f'{column_str} histogram');
```

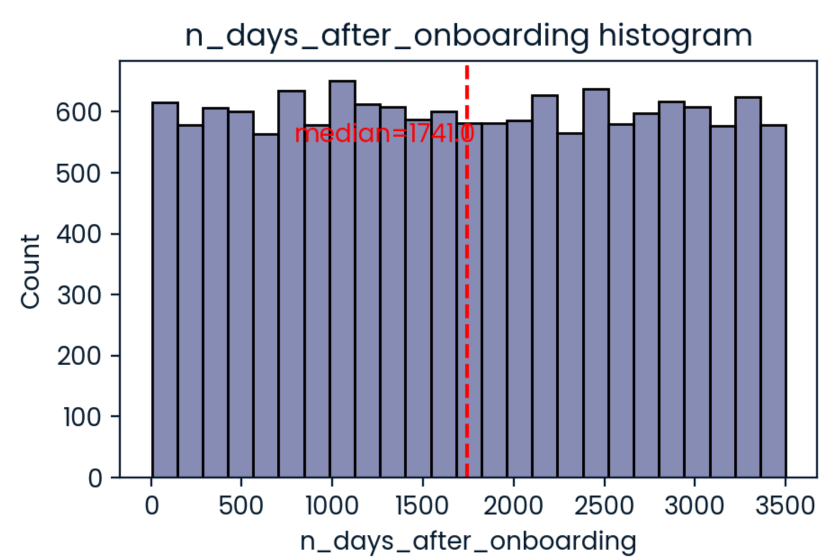
```python
histogrammer('total_sessions')
```



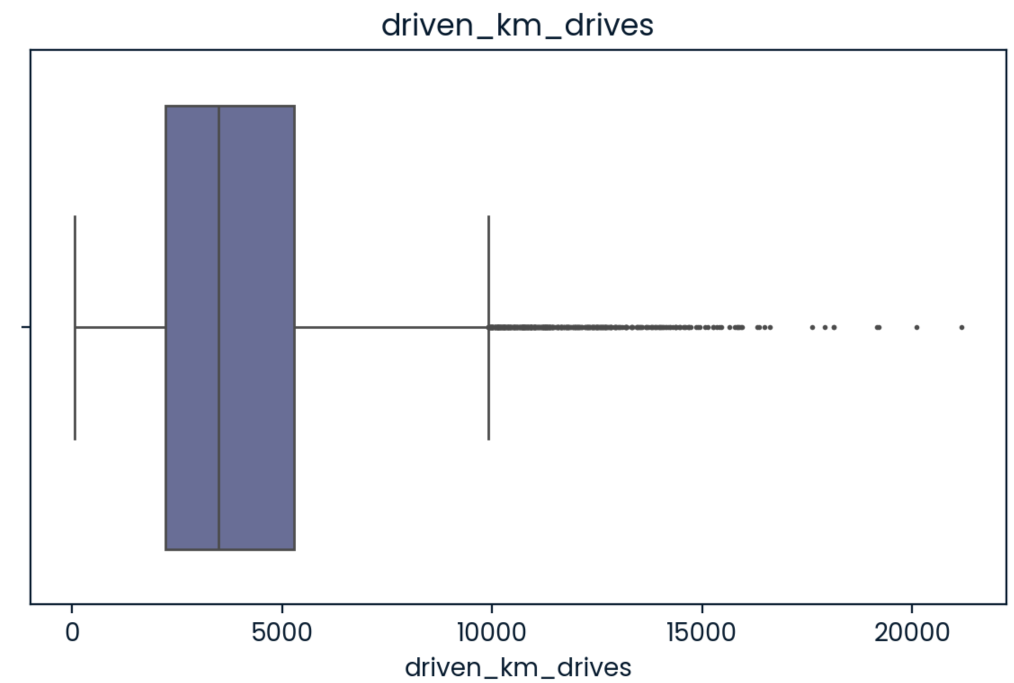
```python
# Box plot
plt.figure(figsize=(7,4))
plt.title('n_days_after_onboarding')
sns.boxplot(x=df['n_days_after_onboarding'],fliersize=1);
```



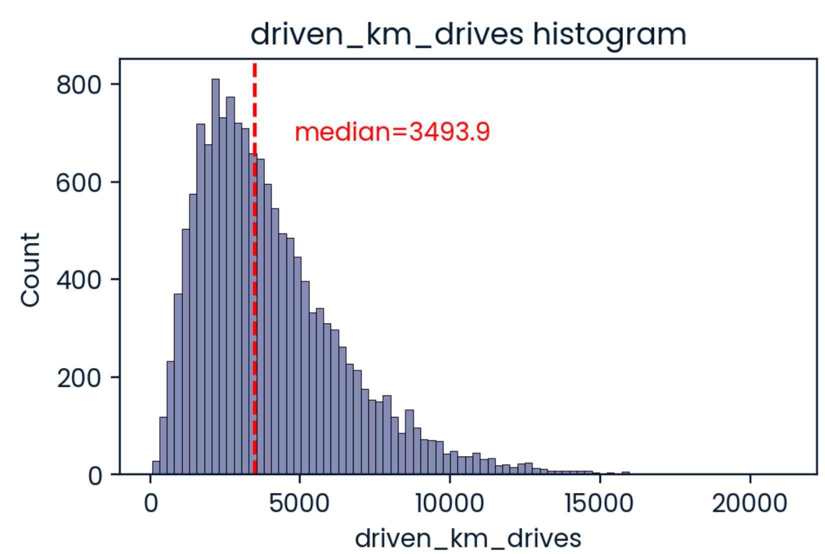

```python
# Histogram
histogrammer('n_days_after_onboarding')
```



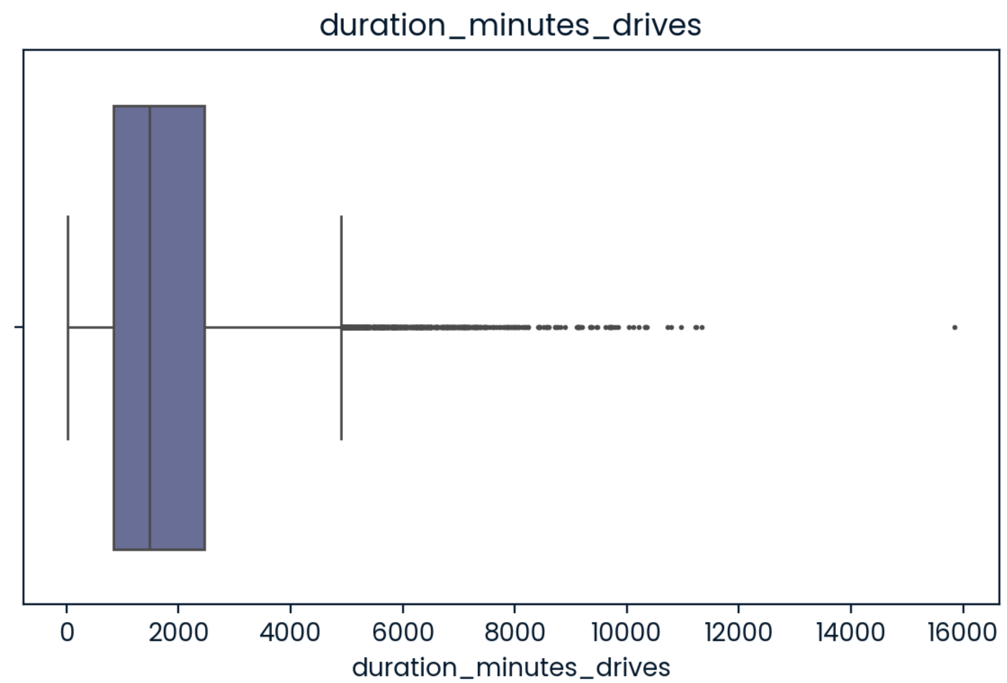
```
# Box plot
plt.figure(figsize=(7,4))
plt.title('driven_km_drives')
sns.boxplot(x=df['driven_km_drives'],fliersize=1);
```



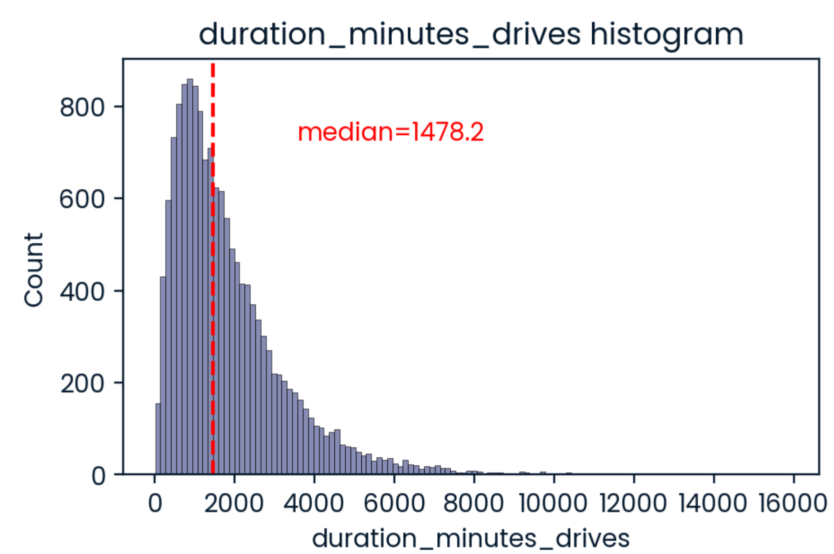

```
# Histogram
histogrammer('driven_km_drives')
```



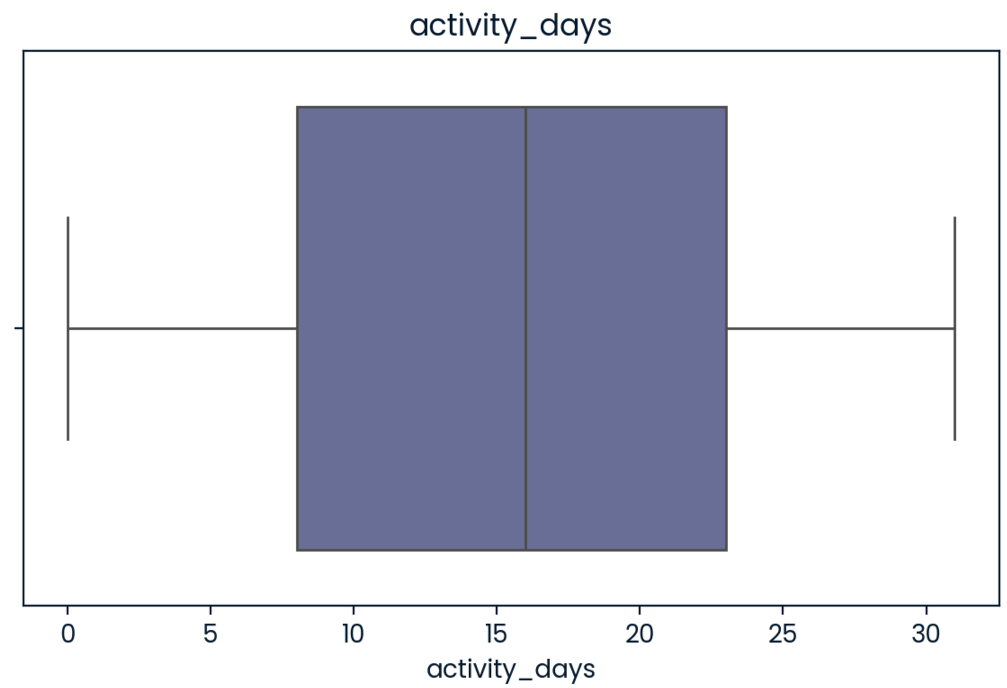
```python
# Box plot
plt.figure(figsize=(7,4))
plt.title('duration_minutes_drives')
sns.boxplot(x=df['duration_minutes_drives'],fliersize=1);
```




```python
# Histogram
histogrammer('duration_minutes_drives')
```

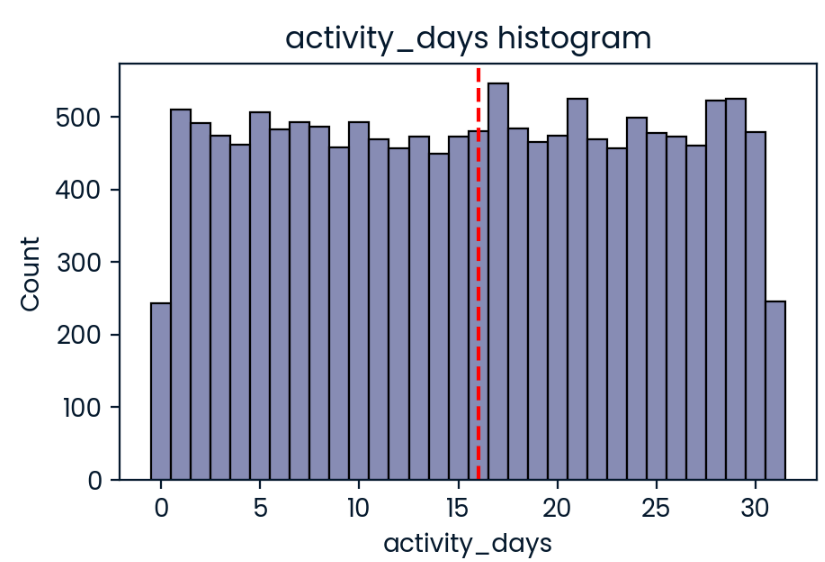


```python
# Box plot
plt.figure(figsize=(7,4))
plt.title('activity_days')
sns.boxplot(x=df['activity_days'],fliersize=1);
```

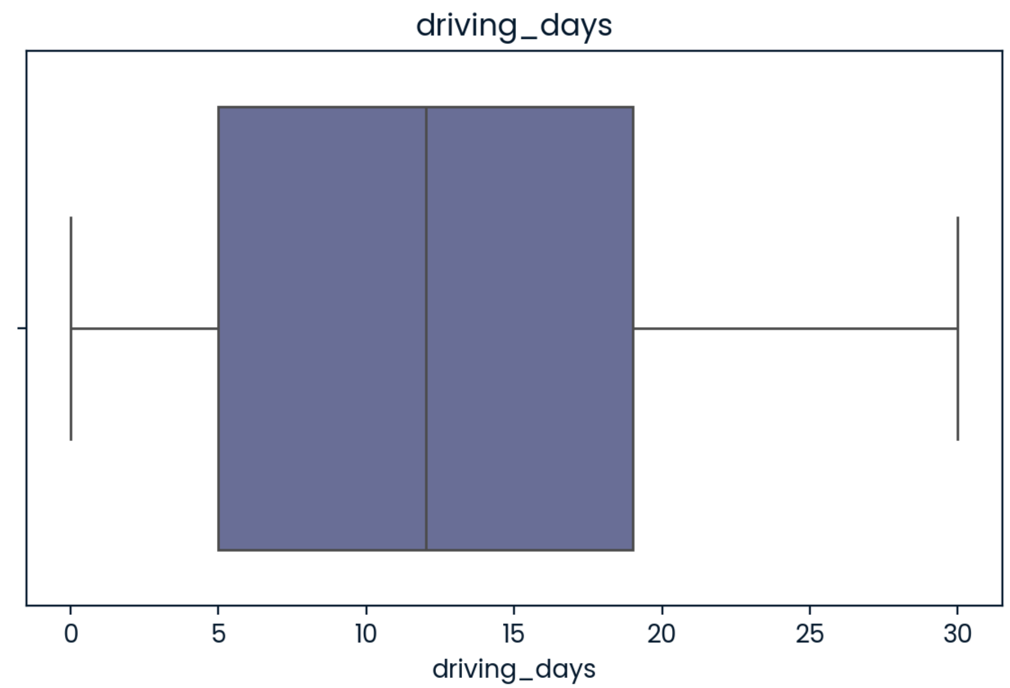



```python
# Histogram
histogrammer('activity_days', median_text=False, discrete=True)
```

```
Median: 16.0
```

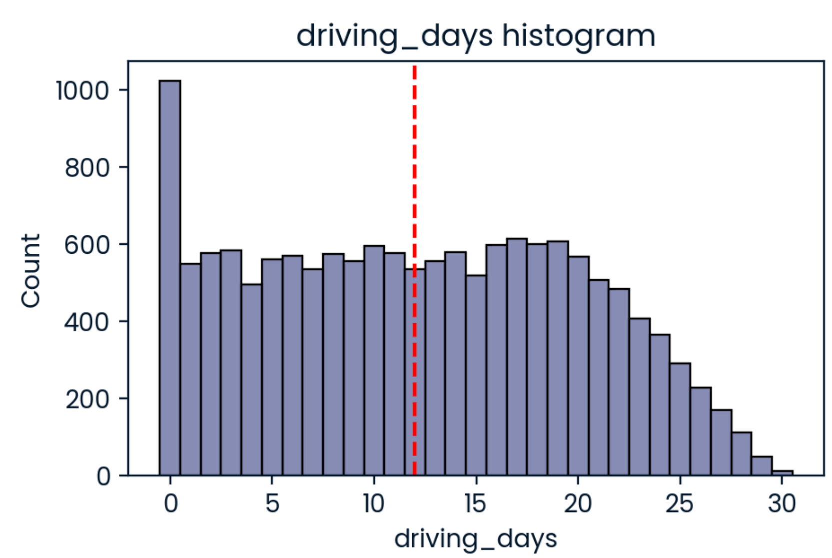


```
# Box plot
plt.figure(figsize=(7,4))
plt.title('driving_days')
sns.boxplot(x=df['driving_days'],fliersize=1);
```

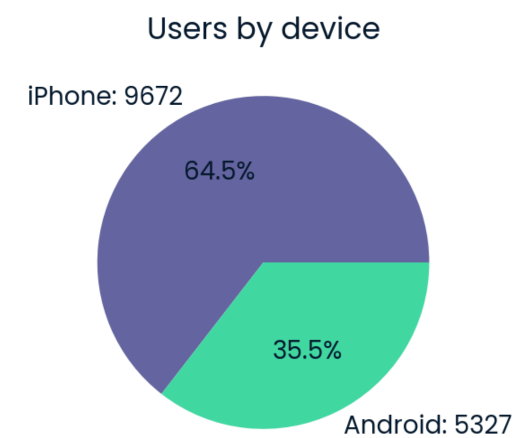



```
# Histogram
histogrammer('driving_days',median_text=False, discrete=True)
```

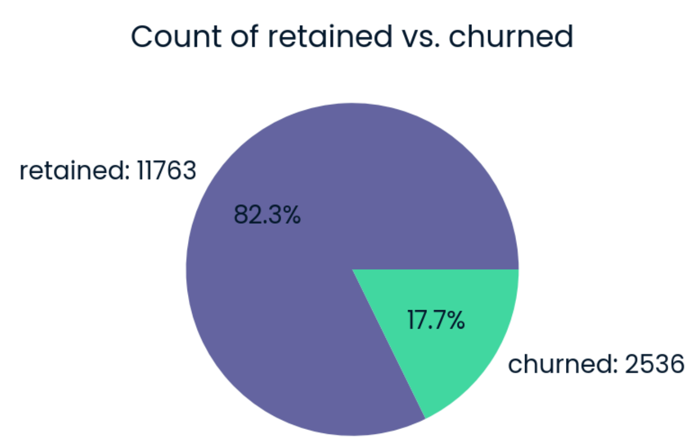
```
Median: 12.0
```



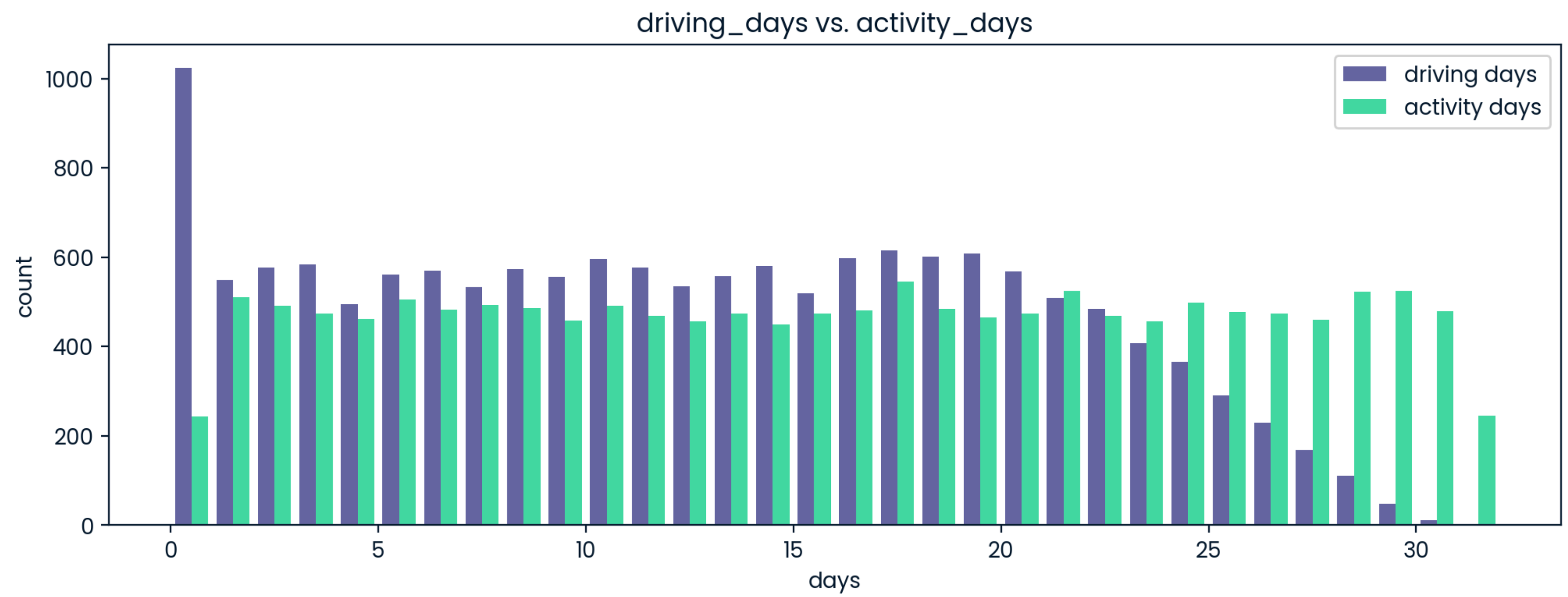
```python
# Pie chart
fig = plt.figure(figsize=(3,3))
data=df['device'].value_counts()
plt.pie(data,
        labels=[f'{data.index[0]}: {data.values[0]}',
                f'{data.index[1]}: {data.values[1]}'],
        autopct='%1.1f%%'
        )
plt.title('Users by device');
```




```python
# Pie chart
fig = plt.figure(figsize=(3,3))
data=df['label'].value_counts()
plt.pie(data,
        labels=[f'{data.index[0]}: {data.values[0]}',
                f'{data.index[1]}: {data.values[1]}'],
        autopct='%1.1f%%'
        )
plt.title('Count of retained vs. churned');
```



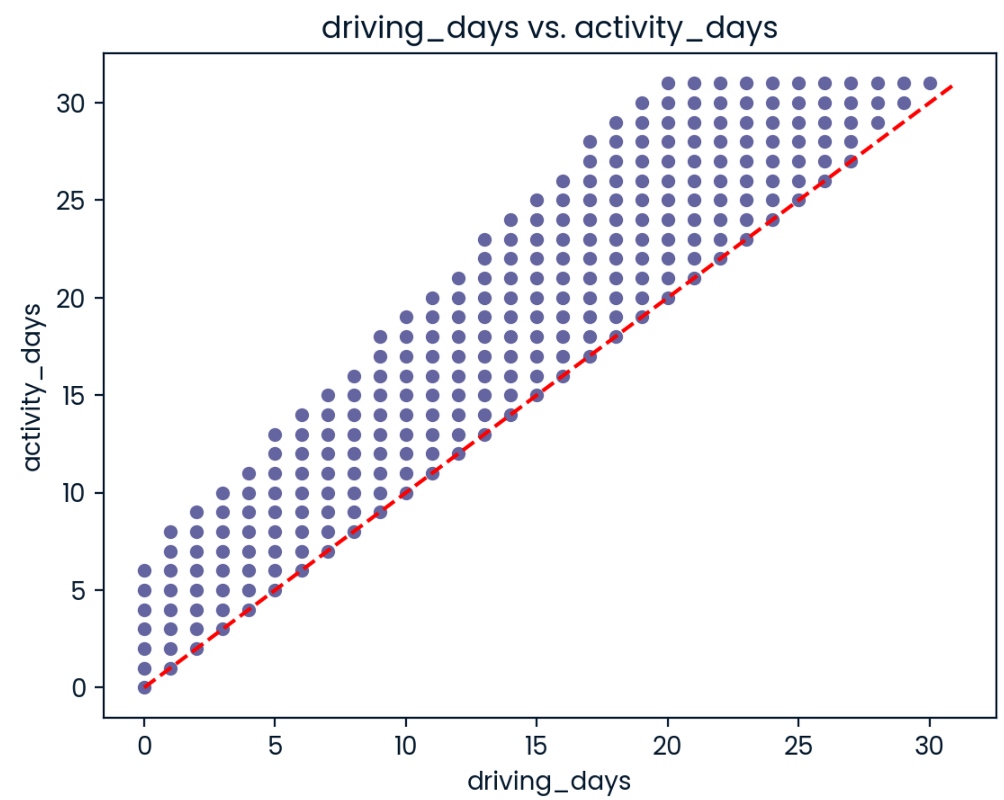
```
# Histogram
plt.figure(figsize=(12,4))
label=['driving days', 'activity days']
plt.hist([df['driving_days'], df['activity_days']],
         bins=range(0,33),
         label=label)
plt.xlabel('days')
plt.ylabel('count')
plt.legend()
plt.title('driving_days vs. activity_days');
```



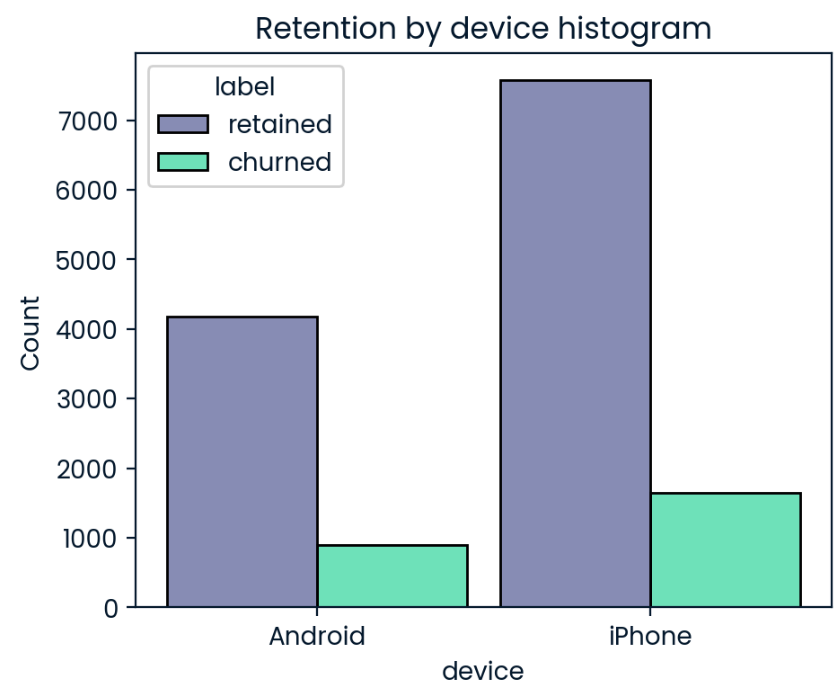

```
print(df['driving_days'].max())
print(df['activity_days'].max())
```

```
30
31
```

```python
# Scatter plot
sns.scatterplot(data=df, x='driving_days', y='activity_days')
plt.title('driving_days vs. activity_days')
plt.plot([0,31], [0,31], color='red', linestyle='--');
```



```python
# Histogram
plt.figure(figsize=(5,4))
sns.histplot(data=df,
             x='device',
             hue='label',
             multiple='dodge',
             shrink=0.9
             )
plt.title('Retention by device histogram');
```




```python
# 1. Create `km_per_driving_day` column
df['km_per_driving_day'] = df['driven_km_drives'] / df['driving_days']

# 2. Call `describe()` on the new column
df['km_per_driving_day'].describe()
```

	km_per_driving...
count	14999
mean	
std	
min	3.022062523
25%	167.28041375
50%	323.1459379
75%	757.9256678
max	

Rows: 8

Expand

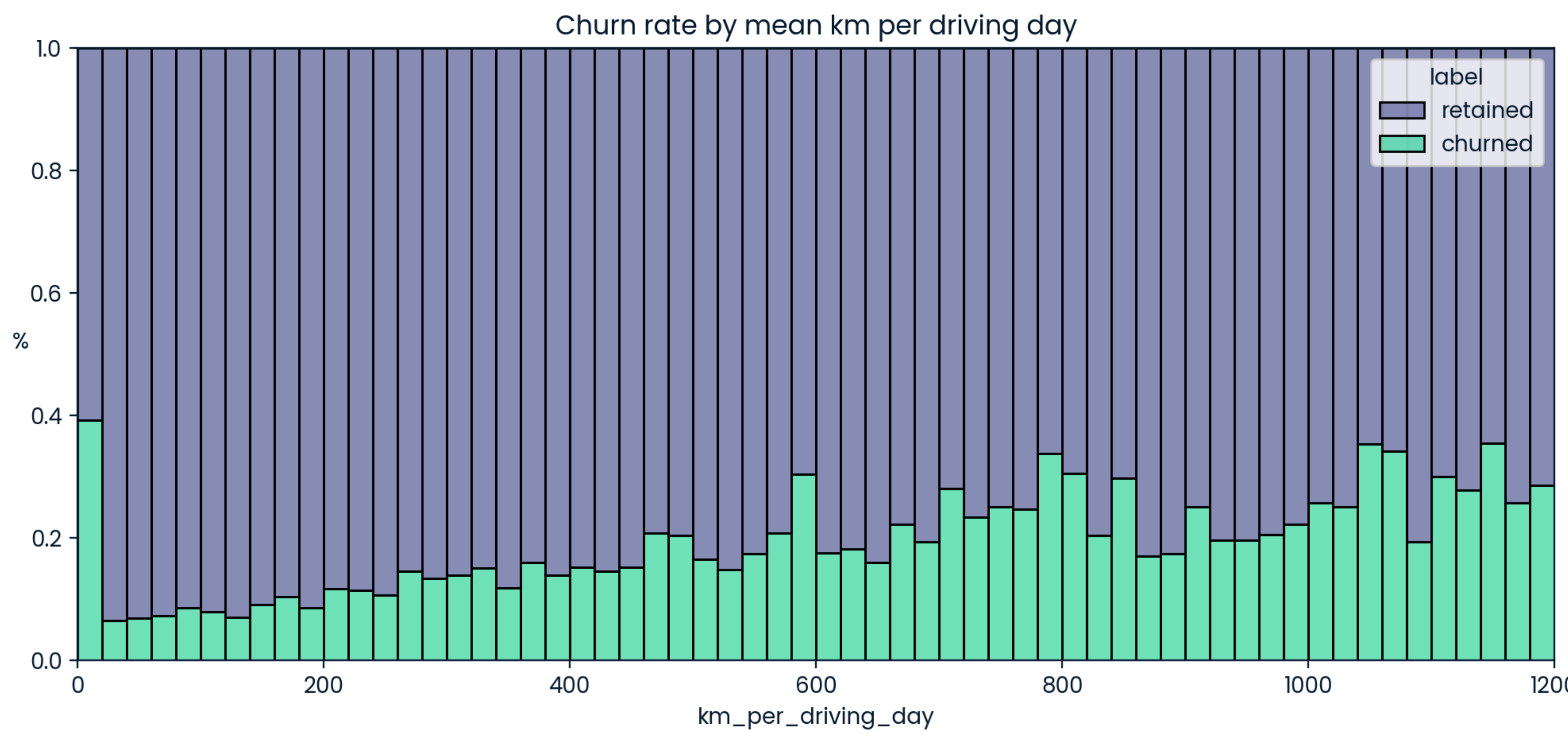
```python
# 1. Convert infinite values to zero
df.loc[df['km_per_driving_day']==np.inf, 'km_per_driving_day'] = 0

# 2. Confirm that it worked
df['km_per_driving_day'].describe()
```

	km_per_driving...
count	14999
mean	578.9631134118
std	1030.0943842273
min	0
25%	136.2388953583
50%	272.889272087
75%	558.6869182125
max	15420.23411

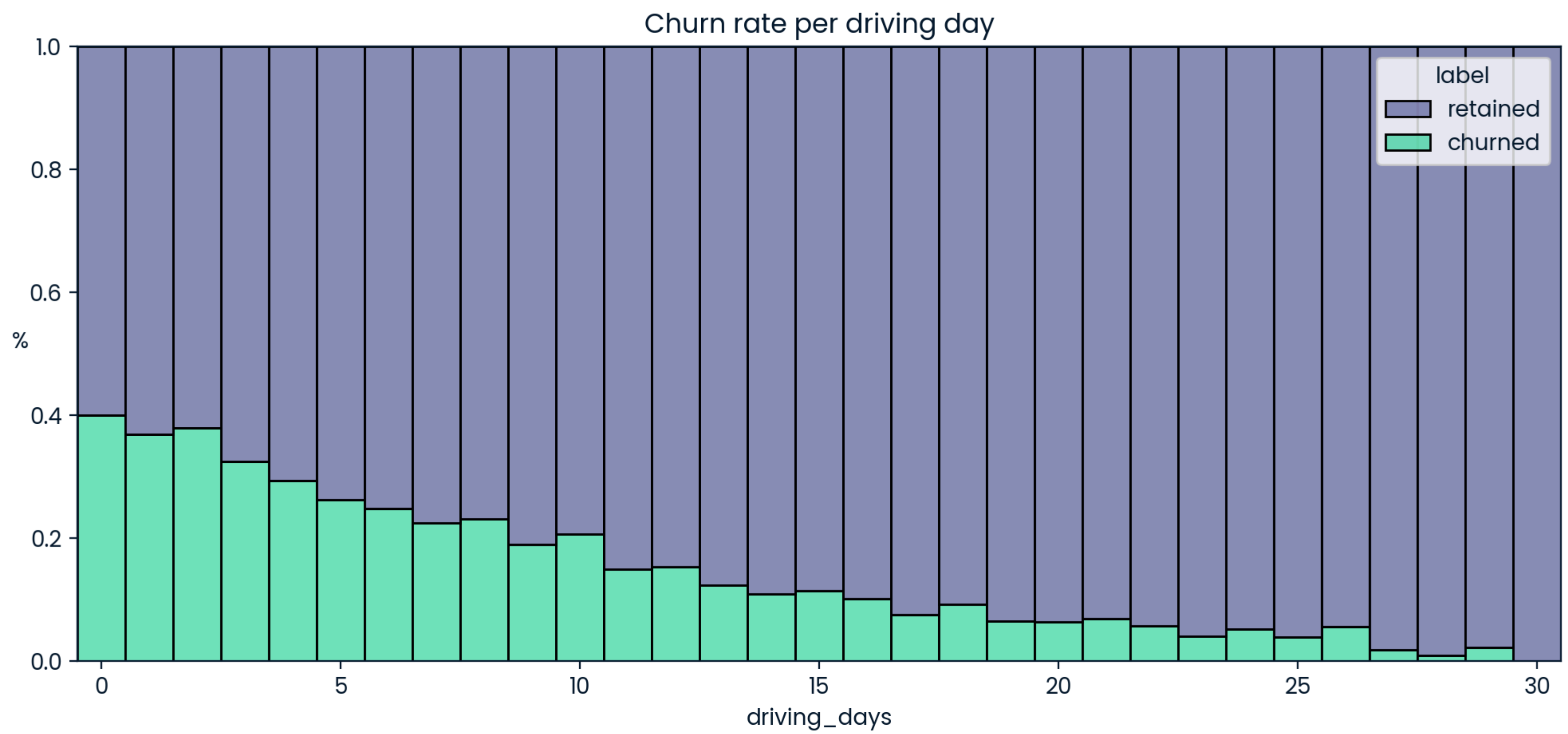
Rows: 8

Expand

```python
# Histogram
plt.figure(figsize=(12,5))
sns.histplot(data=df,
             x='km_per_driving_day',
             bins=range(0,1201,20),
             hue='label',
             multiple='fill')
plt.ylabel('%', rotation=0)
plt.title('Churn rate by mean km per driving day');
```



```python
# Histogram
plt.figure(figsize=(12,5))
sns.histplot(data=df,
             x='driving_days',
             bins=range(1,32),
             hue='label',
             multiple='fill',
             discrete=True)
plt.ylabel('%', rotation=0)
plt.title('Churn rate per driving day');
```



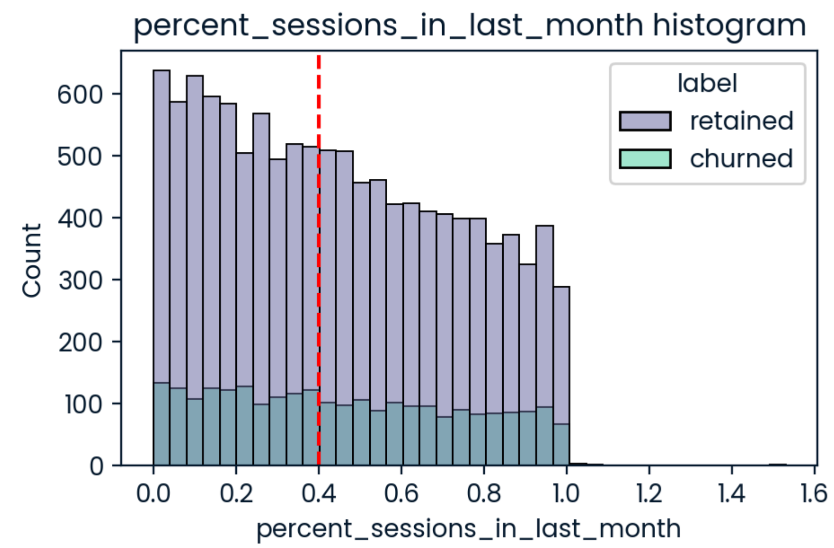

```python
df['percent_sessions_in_last_month'] = df['sessions'] / df['total_sessions']
```

```python
df['percent_sessions_in_last_month'].median()
```

```
np.float64(0.42309702992763176)
```

```
# Histogram
histogrammer('percent_sessions_in_last_month',
             hue=df['label'],
             multiple='layer',
             median_text=False)
```

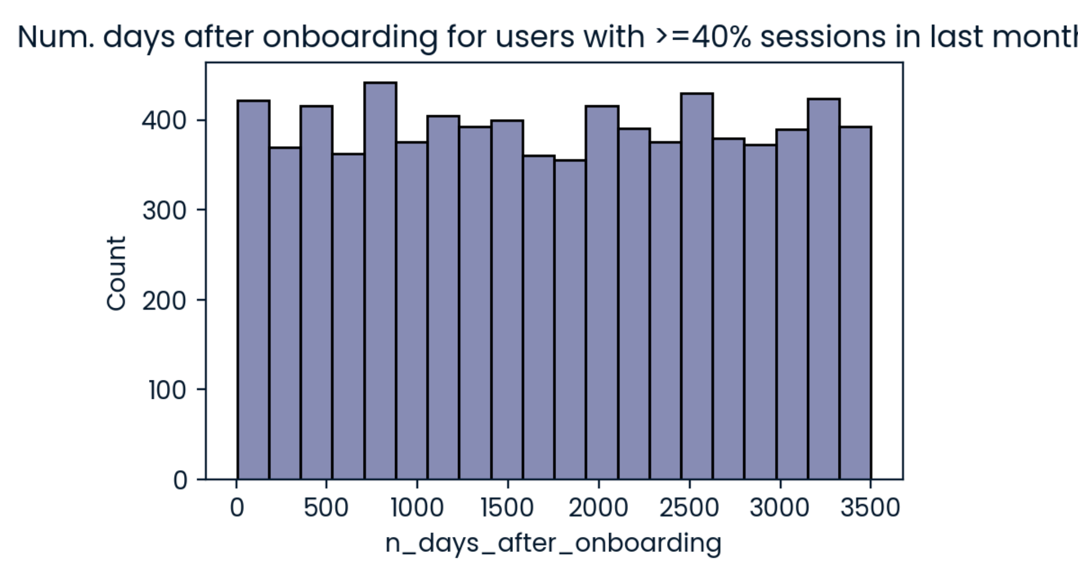
```
Median: 0.4
```




```
df['n_days_after_onboarding'].median()
```

```
np.float64(1741.0)
```

```
# Histogram
data = df.loc[df['percent_sessions_in_last_month']>=0.4]
plt.figure(figsize=(5,3))
sns.histplot(x=data['n_days_after_onboarding'])
plt.title('Num. days after onboarding for users with >=40% sessions in last month');
```



```python
def outlier_imputer(column_name, percentile):
    # Calculate threshold
    threshold = df[column_name].quantile(percentile)
    # Impute threshold for values > than threshold
    df.loc[df[column_name] > threshold, column_name] = threshold

    print('{:>25} | percentile: {} | threshold: {}'.format(column_name, percentile, threshold))
```

```python
for column in ['sessions', 'drives', 'total_sessions',
               'driven_km_drives', 'duration_minutes_drives']:
               outlier_imputer(column, 0.95)
```

```
                 sessions | percentile: 0.95 | threshold: 243.0
                   drives | percentile: 0.95 | threshold: 201.0
           total_sessions | percentile: 0.95 | threshold: 454.3632037399997
         driven_km_drives | percentile: 0.95 | threshold: 8889.7942356
  duration_minutes_drives | percentile: 0.95 | threshold: 4668.899348999998
```

```python
df.describe()
```

	ID	sessions	drives	total_se...	n_days_after_onboardi...	total_navigations_fav1	total_navigations_fav2	driven_km_...	duration_minutes_drives
count	14999	14999	14999	14999	14999	14999	14999	14999	1499
mean	7499	76.5687045803	64.0582038803	184.0313202355	1749.8377891859	121.6059737316	29.6725115008	3939.6327635213	1789.64742611
std	4329.9826789492	67.2979578592	55.3069241595	118.6004625602	1008.5138761173	148.1215436251	45.3946507312	2216.0415103772	1222.7051669
min	0	0	0	0.2202109438	4	0	0	60.44125046	18.2820824
25%	3749.5	23	20	90.66115631	878	9	0	2212.6006065	835.996259
50%	7499	56	48	159.5681147	1741	71	9	3493.858085	1478.24985
75%	11248.5	112	93	254.19234075	2623.5	178	43	5289.861262	2464.36263
max	14998	243	201	454.36320374	3500	1236	415	8889.7942356	4668.89934

Rows: 8

Expand

```python
df['monthly_drives_per_session_ratio'] = (df['drives']/df['sessions'])
df.head(10)
```

					total_se...	n_days_after_onboardi...	total_navigations_fav1	total_navigations_fav2	driven_km_...	duration_minutes_drives	activit...
0	0	retained	243	201	296.7482729	2276	208	0	2628.845068	1985.775061	
1	1	retained	133	107	326.8965962	1225	19	64	8889.7942356	3160.472914	
2	2	retained	114	95	135.5229263	2651	0	0	3059.148818	1610.735904	
3	3	retained	49	40	67.58922127	15	322	7	913.5911231	587.1965423	
4	4	retained	84	68	168.2470201	1562	166	5	3950.202008	1219.555924	
5	5	retained	113	103	279.5444373	2637	0	0	901.2386985	439.1013973	
6	6	retained	3	2	236.7253142	360	185	18	5249.172828	726.5772047	
7	7	retained	39	35	176.0728446	2999	0	0	7892.052468	2466.981741	
8	8	retained	57	46	183.5320177	424	0	26	2651.709764	1594.342984	
9	9	churned	84	68	244.8021148	2997	72	0	6043.460295	2341.838528	

Rows: 10

Expand

Questions:

What types of distributions did you notice in the variables? What did this tell you about the data? Nearly all the variables were either very right-skewed or uniformly distributed. For the right-skewed distributions, this means that most users had values in the lower end of the range for that variable. For the uniform distributions, this means that users were generally equally likely to have values anywhere within the range for that variable.

Was there anything that led you to believe the data was erroneous or problematic in any way? Most of the data was not problematic, and there was no indication that any single variable was completely wrong. However, several variables had highly improbable or perhaps even impossible outlying values, such as driven_km_drives. Some of the monthly variables also might be problematic, such as activity_days and driving_days, because one has a max value of 31 while the other has a max value of 30, indicating that data collection might not have occurred in the same month for both of these variables.

Did your investigation give rise to further questions that you would like to explore or ask the Waze team about? Yes. I'd want to ask the Waze data team to confirm that the monthly variables were collected during the same month, given the fact that some have max values of 30 days while others have 31 days. I'd also want to learn why so many long-time users suddenly started using the app so much in just the last month. Was there anything that changed in the last month that might prompt this kind of behavior?

What percentage of users churned and what percentage were retained? Less than 18% of users churned, and ~82% were retained.

What factors correlated with user churn? How? Distance driven per driving day had a positive correlation with user churn. The farther a user drove on each driving day, the more likely they were to churn. On the other hand, number of driving days had a negative correlation with churn. Users who drove more days of the last month were less likely to churn.

Did newer uses have greater representation in this dataset than users with longer tenure? How do you know? No. Users of all tenures from brand new to ~10 years were relatively evenly represented in the data. This is borne out by the histogram for n_days_after_onboarding, which reveals a uniform distribution for this variable.